

Assignment No. 08: Coverage Generation and Analysis using Synopsys VCS

Q1. Perform functional coverage analysis for a 4-bit Universal Shift Register using Synopsys VCS.

- a) Develop a testbench that verifies all operating modes.
- b) Enable Line, Toggle, Branch, and Condition coverage during compilation.
- c) Generate the coverage database and coverage report.
- d) Analyze the generated report and identify any uncovered statements.

DESIGN CODE:

```
module usr_4bit (  
    input wire clk,  
    input wire rst_n,  
    input wire [1:0] select,  
    input wire shift_left_in,  
    input wire shift_right_in,  
    input wire [3:0] d,  
    output reg [3:0] q    );  
  
    always @(posedge clk or negedge rst_n) begin  
        if (!rst_n) begin  
            q <= 4'b0000;  
        end else begin  
            case (select)  
                2'b00: q <= q;  
                2'b01: q <= {shift_right_in, q[3:1]};  
                2'b10: q <= {q[2:0], shift_left_in};  
                2'b11: q <= d;  
                default: q <= q;  
            endcase  
        end  
    end  
  
endmodule
```

TESHBENCH CODE:

```
`timescale 1ns/1ps  
  
module tb_usr_4bit;  
  
    reg clk;  
    reg rst_n;  
    reg [1:0] select;  
    reg shift_left_in;  
    reg shift_right_in;  
    reg [3:0] d;
```

```

wire [3:0] q;

usr_4bit uut (
    .clk(clk),
    .rst_n(rst_n),
    .select(select),
    .shift_left_in(shift_left_in),
    .shift_right_in(shift_right_in),
    .d(d),
    .q(q));

always #10 clk = ~clk;

initial begin

    $dumpfile("usr_simulation.vcd");
    $dumpvars(0, tb_usr_4bit);

    clk = 0;
    rst_n = 0;
    select = 2'b00;
    shift_left_in = 0;
    shift_right_in = 0;
    d = 4'b0000;

    #15 rst_n = 1;

    #10;
    select = 2'b11;
    d = 4'b1010;

    #20;
    select = 2'b00;
    d = 4'b1111;

    #20;
    select = 2'b01;
    shift_right_in = 1;
    #20;
    shift_right_in = 0;
    #40;

    #20;
    select = 2'b10;
    shift_left_in = 1;
    #20;
    shift_left_in = 0;

```

```

#40;
$display("Simulation complete. Waveform file generated.");
$finish;

end
endmodule

```

VCD FILE :

```

Compiler version W-2024.09; Runtime version W-2024.09; Jul 17 20:44 2026
[SCL] SCL trace is enabled.
Checkout succeeded: VCS-BASE-RUNTIME/AF3AD4D7787CF4828637
License file: 27020@14.139.1.126
License Server: 27020@14.139.1.126
[SCL] 07/17/2026 20:44:34 Checking status for feature VCS-BASE-RUNTIME
[SCL] 07/17/2026 20:44:34 PID:210911 Client:smartlab6.calicut.nielit.in Server:27020@14.139.1.126 Checkout succ
RUNTIME 2024.09 from VCS-Base-Runtime-Pkg
 0 | 0 | 00 | 0 | 0 | 0000 | 0000
15 | 1 | 00 | 0 | 0 | 0000 | 0000
25 | 1 | 11 | 0 | 0 | 1010 | 0000
30 | 1 | 11 | 0 | 0 | 1010 | 1010
45 | 1 | 00 | 0 | 0 | 1111 | 1010
65 | 1 | 01 | 0 | 1 | 1111 | 1010
70 | 1 | 01 | 0 | 1 | 1111 | 1101
85 | 1 | 01 | 0 | 0 | 1111 | 1101
90 | 1 | 01 | 0 | 0 | 1111 | 0110
110 | 1 | 01 | 0 | 0 | 1111 | 0011
130 | 1 | 01 | 0 | 0 | 1111 | 0001
145 | 1 | 10 | 1 | 0 | 1111 | 0001
150 | 1 | 10 | 1 | 0 | 1111 | 0011
165 | 1 | 10 | 0 | 0 | 1111 | 0011
170 | 1 | 10 | 0 | 0 | 1111 | 0110
190 | 1 | 10 | 0 | 0 | 1111 | 1100
Simulation complete.Waveform file generated
$finish called from file "../RTL/TB/tb_usr_4bit.v", line 66.
$finish at simulation time 205000
V C S   S i m u l a t i o n   R e p o r t
Time: 205000 ps
CPU Time: 0.250 seconds; Data structure size: 0.0Mb
Fri Jul 17 20:44:34 2026

```

Line > Toggle >

Line Coverage for Module : tb_usr_4bit

	Line No.	Total	Covered	Percent
TOTAL		41	41	100.00
ALWAYS	23	2	2	100.00
INITIAL	25	1	1	100.00
INITIAL	29	38	38	100.00

```

22
23      2/2      always #10 clk =~clk;
24      initial begin
25      1/1      $monitor("%10d | %b | %b | %b | %b | %b | %b", $time,r
26      end
27
28      initial begin
29      1/1      $dumpfile("usr_simlation.vcd");
30      1/1      $dumpvars(0,tb_usr_4bit);
31
32      1/1      clk = 0;
33      1/1      rst_n=0;

```

Module Definition

dashboard | hierarchy | modlist | groups | tests | asserts

Line Toggle

Line Coverage for Module : usr_4bit

	Line No.	Total	Covered	Percent
TOTAL		8	7	87.50
ALWAYS	13	8	7	87.50

```
12          always @(posedge clk or negedge rst_n)begin
13              1/1          if (!rst_n)begin
14                  1/1          q <= 4'b0000;
15              end else begin
16                  1/1          case (select)
17                      1/1          2'b00 :q <=q;
18                      1/1          2'b01 :q <={shift_right_in,q[3:1]};
19                      1/1          2'b10 :q <={q[2:0], shift_left_in};
20                      1/1          2'b11 :q <=d;
21              0/1          ==>          default: q <=q;
```

Toggle Coverage for Module : usr_4bit

Total	Covered	Percent
-------	---------	---------

Applications : Unified Coverage Report... Coverage Sun 19 Jul, 20:34 int-05

Unified Coverage Report :: x +

file:///home/int-05/Desktop/counter/Simulation/Coverage/mod0.html#inst_tag_0

Customer Portal Red Hat Red Hat Products D... Red Hat Enterprise ... Red Hat Developer ... Red Hat Container ... Red Hat Hybrid Clo...

SYNOPSYS

Module Definition

dashboard | hierarchy | modlist | groups | tests | asserts

tb_usr_4bit.uut 84.82 87.50 82.14

Module Instance : tb_usr_4bit.uut

Instance :

SCORE	LINE	COND	TOGGLE
84.82	87.50		82.14

Instance's subtree :

SCORE	LINE	COND	TOGGLE
84.82	87.50		82.14

Parent :

SCORE	LINE	COND	TOGGLE	NAME
91.07	100.00		82.14	tb_usr_4bit

Subtrees :

NAME	SCORE	LINE	COND	TOGGLE
no children				

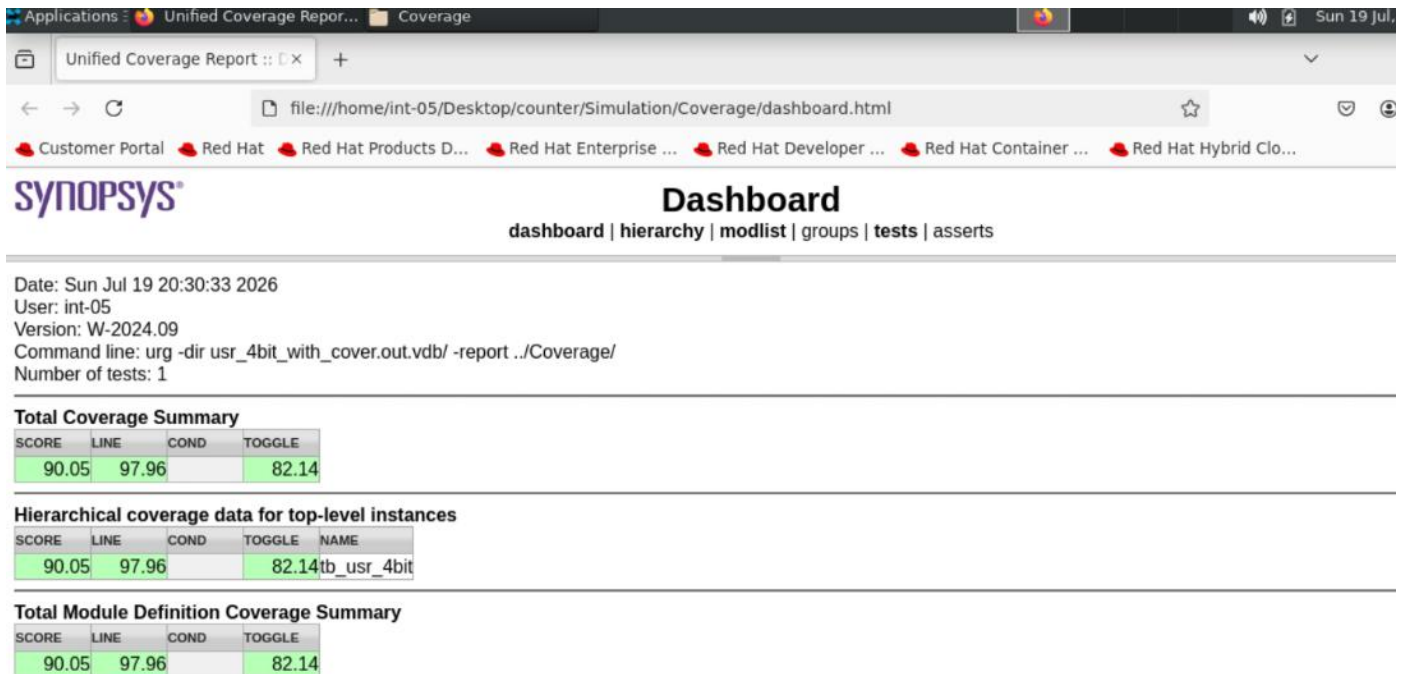
Since this is the module's only instance, the coverage report is the same as for the module.

Line Toggle

	Total	Covered	Percent
Totals	7	5	71.43
Total Bits	28	23	82.14
Total Bits 0->1	14	14	100.00
Total Bits 1->0	14	9	64.29
Ports	7	5	71.43
Port Bits	28	23	82.14
Port Bits 0->1	14	14	100.00
Port Bits 1->0	14	9	64.29

Port Details

Name	Toggle	Toggle 1->0	Toggle 0->1	Direction
clk	Yes	Yes	Yes	INPUT
rst_n	No	No	Yes	INPUT
select[1:0]	Yes	Yes	Yes	INPUT
shift_left_in	Yes	Yes	Yes	INPUT
shift_right_in	Yes	Yes	Yes	INPUT
d[3:0]	No	No	Yes	INPUT
q[3:0]	Yes	Yes	Yes	OUTPUT



- Q2. Perform functional coverage analysis for an 8-bit Arithmetic Logic Unit (ALU) using Synopsys VCS.
- a) Develop test cases to verify all arithmetic and logical operations.
 - b) Generate Line, Toggle, Branch, and Condition coverage reports.
 - c) Identify operations that are not fully covered.
 - d) Modify the testbench to improve the overall functional coverage.
 - e) Compare the initial and updated coverage report.

DESIGN CODE:

```
`timescale 1ns/1ps
module alu_8bit (
    input wire [7:0] a,
    input wire [7:0] b,
    input wire [2:0] opcode,
    output reg [7:0] alu_out,
    output reg      carry_out );

    always @(*) begin

        alu_out    = 8'b0000_0000;
        carry_out = 1'b0;

        case (opcode)
            3'b000: begin
                {carry_out, alu_out} = a + b;
            end
        end
    end
```

```

        3'b001: begin
            alu_out = a - b;
            carry_out = (a < b);
        end
        3'b010: begin
            alu_out = a & b;
        end
        3'b011: begin
            alu_out = a | b;
        end
        3'b100: begin
            alu_out = a ^ b;
        end
        3'b101: begin
            alu_out = a << 1;
        end
        3'b110: begin
            alu_out = a >> 1;
        end
        3'b111: begin
            alu_out = a;
        end
        default: begin
            alu_out = a;
        end
    endcase
end
endmodule

```

TESTBENCH CODE:

```

`timescale 1ns/1ps

module tb_alu_8bit;

    reg [7:0] a;
    reg [7:0] b;
    reg [2:0] opcode;
    wire [7:0] alu_out;
    wire carry_out;

    alu_8bit uut (
        .a(a),
        .b(b),
        .opcode(opcode),
        .alu_out(alu_out),
        .carry_out(carry_out) );

```

```

initial begin
    $display("\n-----");
    $display("    Time(ns) | Opcode |    Input A    |    Input B    | Out
Result | Carry");
    $display("-----");
    $monitor("%11d |    %b |    %b |    %b |    %b |    %b",
            $time, opcode, a, b, alu_out, carry_out);
end

```

```

initial begin

    $dumpfile("alu_simulation.vcd");
    $dumpvars(0, tb_alu_8bit);

    a = 8'h00; b = 8'h00; opcode = 3'b111;

    #10;
    opcode = 3'b000;
    a = 8'hF0; b = 8'h0F;
    #10;
    a = 8'hFF; b = 8'h01;

    #10;
    opcode = 3'b001;
    a = 8'h45; b = 8'h10;
    #10;
    a = 10;    b = 20;

    #10;
    opcode = 3'b010;
    a = 8'b1100_1010; b = 8'b1010_0101;

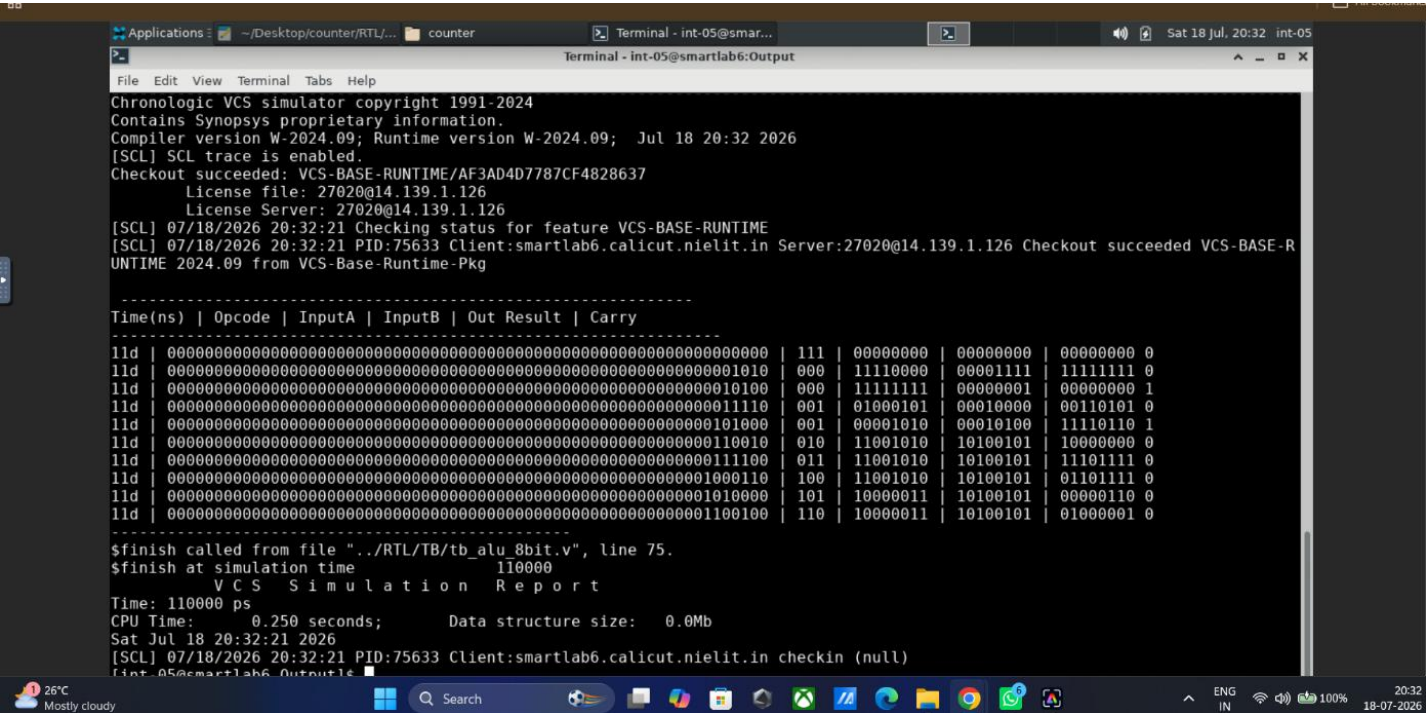
    #10;
    opcode = 3'b011;
    a = 8'b1100_1010; b = 8'b1010_0101;

    #10;
    opcode = 3'b100;
    a = 8'b1100_1010; b = 8'b1010_0101;

    #10;
    opcode = 3'b101;
    a = 8'b1000_0011;

```


VCD FILE :



Synopsys

Dashboard

dashboard | hierarchy | modlist | groups | tests | asserts

Date: Sun Jul 19 20:51:55 2026
User: int-05
Version: W-2024.09
Command line: urg -dir alu_8bit_with_cover.out.vdb/ -report ../Coverage/
Number of tests: 1

Total Coverage Summary

SCORE	LINE	COND	TOGGLE
95.74	98.63		92.86

Hierarchical coverage data for top-level instances

SCORE	LINE	COND	TOGGLE	NAME
95.74	98.63		92.86	tb_alu_8bit

Total Module Definition Coverage Summary

SCORE	LINE	COND	TOGGLE
95.74	98.63		92.86

file:///home/int-05/Desktop/counter/Simulation/Coverage/mod0.html#inst_tag_0

Customer Portal Red Hat Red Hat Products D... Red Hat Enterprise ... Red Hat Developer ... Red Hat Container ... Red Hat Hybrid Clo...

synopsys

Module Definition

dashboard | hierarchy | modlist | groups | tests | asserts

Module self-instances :

NAME	SCORE	LINE	COND	TOGGLE
tb_alu_8bit	96.43	100.00		92.86

Module Instance : tb_alu_8bit

Instance :

SCORE	LINE	COND	TOGGLE
96.43	100.00		92.86

Instance's subtree :

SCORE	LINE	COND	TOGGLE
95.74	98.63		92.86

Parent :
none

Subtrees :

NAME	SCORE	LINE	COND	TOGGLE
uut	92.58	92.31		92.86

Since this is the module's only instance, the coverage report is the same as for the module.

Line Toggle

Line Coverage for Module : tb_alu_8bit

	Line No.	Total	Covered	Percent
TOTAL		60	60	100.00
INITIAL	19	4	4	100.00
INITIAL	26	56	56	100.00

18

19

20

21

22

23

24

25

26

27

28

29

30

31

initial begin

\$display("\n -----

\$display("Time(ns) | Opcode | InputA | InputB | Out Result | C

\$display("-----

\$monitor("11d | %b | %b | %b | %b | %b ", \$time,opcode,a,b,alu

end

initial begin

\$dumpfile("alu_simulation.vcd");

\$dumpvars(0,tb_alu_8bit);

a = 8'h00; b=8'h00; opcode =3'b111;

#10;

synopsys

Module Definition

dashboard | hierarchy | modlist | groups | tests | asserts

tb_alu_8bit.uut

92.58

92.31

92.86

Module Instance : tb_alu_8bit.uut

Instance :

SCORE	LINE	COND	TOGGLE
92.58	92.31		92.86

Instance's subtree :

SCORE	LINE	COND	TOGGLE
92.58	92.31		92.86

Parent :

SCORE	LINE	COND	TOGGLE	NAME
96.43	100.00		92.86	tb_alu_8bit

Subtrees :

NAME	SCORE	LINE	COND	TOGGLE
no children				

Since this is the module's only instance, the coverage report is the same as for the module.

Line Toggle

end

default : begin

0/1 ==> alu_out=a;

Toggle Coverage for Module : alu_8bit

	Total	Covered	Percent
Totals	5	4	80.00
Total Bits	56	52	92.86
Total Bits 0->1	28	27	96.43
Total Bits 1->0	28	25	89.29
Ports	5	4	80.00
Port Bits	56	52	92.86
Port Bits 0->1	28	27	96.43
Port Bits 1->0	28	25	89.29

Port Details

Name	Toggle	Toggle 1->0	Toggle 0->1	Direction
a[7:0]	Yes	Yes	Yes	INPUT
b[4:0]	Yes	Yes	Yes	INPUT

dashboard | hierarchy | modlist | groups | tests | asserts

tb_alu_8bit.uut

92.58

92.31

92.86

Module Instance : tb_alu_8bit.uut

Instance :

SCORE	LINE	COND	TOGGLE
92.58	92.31		92.86

Instance's subtree :

SCORE	LINE	COND	TOGGLE
92.58	92.31		92.86

Parent :

SCORE	LINE	COND	TOGGLE	NAME
96.43	100.00		92.86	tb_alu_8bit

Subtrees :

NAME	SCORE	LINE	COND	TOGGLE
no children				

Since this is the module's only instance, the coverage report is the same as for the module.

Line Toggle

Totals

	Total	Covered	Percent
Totals	5	4	80.00
Total Bits	56	52	92.86
Total Bits 0->1	28	27	96.43
Total Bits 1->0	28	25	89.29
Ports	5	4	80.00
Port Bits	56	52	92.86
Port Bits 0->1	28	27	96.43
Port Bits 1->0	28	25	89.29

Port Details

Name	Toggle	Toggle 1->0	Toggle 0->1	Direction
a[7:0]	Yes	Yes	Yes	INPUT
b[4:0]	Yes	Yes	Yes	INPUT
b[5]	No	No	Yes	INPUT
b[6]	No	No	No	INPUT
b[7]	No	No	Yes	INPUT
opcode[2:0]	Yes	Yes	Yes	INPUT
alu_out[7:0]	Yes	Yes	Yes	OUTPUT
carry_out	Yes	Yes	Yes	OUTPUT